

UNIVERSIDADE D
COIMBRA



Sistemas Distribuídos

Licenciatura em Engenharia Informática

Googol

Motor de pesquisa de páginas web distribuído

Relatório de Projeto
Meta 2

Diogo Rodrigues | **2022257625**

Guilherme Rosmaninho | **2022257636**

PL6

2025/2026

1. Índice

1. Índice 2
2. Introdução 3
3. Arquitetura de Software 3
    3.1. Arquitetura MVC (Model-View-Controller) 3
    3.1.1. Model (Modelo): 3
    3.1.2. View (Vista): 4
    3.1.3. Controller (Controlador): 4
    3.2. Organização do código 5
4. Integração com o servidor RPC/RMI (Meta 1) 6
    4.1. Arquitetura de integração 6
    4.2. GatewayService: Cliente RMI 7
    4.3. Fluxo de uma pesquisa 7
    4.4. Tratamento de falhas RMI 8
5. WebSockets e notificações em tempo real 8
    5.1. Configuração WebSocket (STOMP) 9
6. Integração com serviços REST externos 10
    6.1.1. Fluxo de indexação das top stories do HackerNews 11
    6.2. API Openrouter.ai 12
    6.2.1. Análise contextualizada dos resultados de uma pesquisa 12
    6.2.1.1. Fluxo do processamento dos pedidos de análise contextualizada 13
    6.2.2. Spellcheck 14
7. Integração do projeto com um domínio web 15
8. Testes de Software 16
9. Distribuição de tarefas 17
10. Conclusão 17

2. Introdução

Este relatório documenta as decisões de arquitetura e implementação da segunda meta do projeto Googol, que consistiu no desenvolvimento de uma interface Web completa para o motor de pesquisa distribuído. O objetivo principal foi criar uma aplicação Web moderna, interativa e totalmente interoperável com o sistema RMI desenvolvido na Meta 1 (Gateway, Barrels e Downloaders).

A solução implementada utiliza Java Spring Boot como framework Web, Thymeleaf como motor de templates, WebSockets para comunicação em tempo real e integra-se com APIs REST externas (HackerNews e OpenRouter.ai). A arquitetura segue o padrão MVC (Model-View-Controller), garantindo separação de responsabilidades, modularidade e facilidade de manutenção.

3. Arquitetura de Software

3.1. Arquitetura MVC (Model-View-Controller)

3.1.1. Model (Modelo):

Os modelos representam os dados do sistema e são partilhados entre a Meta 1 e a Meta 2:

- Classes do domínio do problema:
  - *SearchResult*: representa um resultado de pesquisa. Constituído por:
    - URL;
    - Título;
    - Snippet - excerto de uma página web;
    - Relevância - considera-se o número de inlinks que uma página possui;
    - Score TF-IDF - métrica utilizada para ranking de páginas.
  - *SearchResponse*: wrapper para a responder a uma pesquisa - contém a lista de resultados e metadados de paginação. Constituído por:
    - Lista de resultados;
    - Metadados de paginação e da pesquisa - úteis para organização da informação para apresentação ao utilizador.
  - *SystemStats*: estatísticas globais do sistema. Constituído por:
    - Número de páginas indexadas;
    - Número de barrels ativos;
    - Estatísticas dos barrels ativos;
    - Top de pesquisas - regista os termos mais pesquisados, de forma não persistente;
  - *BarrelStats*: representam o estado de cada barrel, constituído por:
    - Barrel ID - identificador do barrel no sistema;

        - Tamanho do índice armazenado - em circunstâncias normais será o mesmo valor que os restantes barrels no sistema, no entanto a presença desta métrica ajuda a monitorizar a saúde do sistema;
        - Tempo médio de resposta;
        - Estado da atividade do barrel;
        - Estado do modo de sincronização do barrel - oferece informação acerca de barris com atividades de sincronização a decorrer;
        - ID do peer a fornecer dados de sincronização;
        - Métrica de progresso de sincronização
    - *PageInfo*: Informação completa sobre uma página indexada;
- Interfaces RMI:
    - *IGateway*: interface remota do Gateway (originário na meta 1)
    - *IStatsListener*: interface para callbacks de estatísticas;

Estes objetos transitam entre o backend RMI e o frontend Web (enquanto cliente RMI), garantindo interoperabilidade total.

3.1.2. View (Vista):

As vistas são implementadas com Thymeleaf, um motor de templates server-side que renderiza HTML de forma dinâmica.

- Templates Principais (*src\frontend\webapp\templates*):
    - index.htm: Página inicial com caixa de pesquisa e seletor de modo de utilização do Googol(pesquisa, indexação, indexação HackerNews);
    - *search.html*: Página de resultados de pesquisa com paginação estilo Google;
    - *links.html*: Página que lista páginas que apontam para um URL;
    - *stats.html*: Página de estatísticas detalhadas do sistema;
    - *fragments/layout.html*: Layout base reutilizável (header, footer, estilos);
- Recursos estáticos (src/main/resources/static/):
    - *css/style.css*: Estilos globais com variáveis CSS para temas;
    - *js/stats-websocket.js*: Cliente WebSocket para estatísticas em tempo real;
    - *js/mode-switch.js*: Lógica de alternância entre modos de utilização do Googol (Search/Index/Index HackerNews);
    - js/app.js: Funcionalidades JavaScript gerais

3.1.3. Controller (Controlador):

Os controladores orquestram as interações entre utilizador, serviços e vistas:

- *SearchController* (Controlador MVC principal):
    - GET /: renderiza página principal;

    - GET /search : Processa pesquisas e renderiza resultados;
    - POST /index : Submete URL para indexação;
    - POST /index-hackernews: Indexa stories do HackerNews;
    - GET /links-to : Mostra inlinks de um URL;
    - GET /stats : Renderiza página de estatísticas;
- *ApiController* (controlador REST):
    - GET /api/search : Endpoint REST para pesquisa;
    - GET /api/index : Endpoint REST para indexação;
    - GET /api/stats : Endpoint REST para obter estatísticas;
    - GET /api/hackernews/stories : Lista top HN stories filtradas por termos;
    - POST /api/hackernews/index: Indexa HN stories via API;
    - POST /api/ai-analysis : Gerar analise AI;
    - POST /api/openrouter/test : Endpoint para testar o serviço openrouter.ai;
    - GET /api//links-to : Mostra inlinks de um URL;
    - GET /api/spell-check : Efetua a verificação ortográfica;
    - GET /api/health : Obtém estatísticas para verificação da saúde da conexão RMI.

3.2. Organização do código

A estrutura do projeto foi organizada para maximizar a coesão e minimizar o acoplamento:

Esquema da estrutura de ficheiros das componentes da Meta 2

```
src/frontend/
├── java/
│   ├── GoogolApplication.java
│   │   └── # Classe principal Spring Boot
│   ├── config/
│   │   └── WebSocketConfig.java
│   │       └── # Configuração STOMP/WebSocket para push de stats em tempo real
│   ├── controllers/
│   │   ├── SearchController.java
│   │   │   └── # Controlador MVC - páginas HTML (/, /search, /stats, /links)
│   │   └── ApiController.java
│   │       └── # Controlador REST - endpoints JSON (/api/search, /api/index, /api/stats)
│   └── services/
│       ├── GatewayService.java
│       │   └── # Cliente RMI - abstrai toda comunicação com Gateway
│       ├── StatsUpdateService.java
│       │   └── # Listener RMI → WebSocket - recebe stats do Gateway e distribui via STOMP
│       ├── HackerNewsService.java
│       │   └── # Cliente REST - integração com API Hacker News
```

```
│       ├── OpenAIService.java
│       │   └── # Cliente REST - integração com OpenRouter.ai para análise IA
│       └── SpellCheckService.java
│           └── # Verificação ortográfica - utiliza OpenAIService
└── webapp/
    ├── keystore.p12
    │   └── # Certificado SSL/TLS para HTTPS
    ├── static/
    │   ├── css/
    │   │   └── style.css
    │   │       └── # Estilos globais, design system, variáveis CSS (dark/light modes)
    │   ├── js/
    │   │   ├── app.js
    │   │   │   └── # Funcionalidades JavaScript gerais (validação, utilitários)
    │   │   ├── stats-websocket.js
    │   │   │   └── # Cliente STOMP/WebSocket para atualizações de stats em tempo real
    │   │   ├── global-stats.js
    │   │   │   └── # Lógica de renderização e atualização de estatísticas na UI
    │   │   ├── mode-switch.js
    │   │   │   └── # Switcher entre modos: Search / Index URL / Index Hacker News
    │   │   └── theme-toggle.js
    │   │       └── # Toggle entre tema dark/light
    │   ├── googol_logo.svg
    │   │   └── # Logo principal da aplicação
    │   └── googol_favicon.svg
    │       └── # Favicon da aplicação
    └── templates/
        ├── fragments/
        │   └── layout.html
        │       └── # Layout base reutilizável (header, footer, imports de CSS/JS)
        ├── index.html
        │   └── # Página inicial - logo, search box, mode selector
        ├── search.html
        │   └── # Página de resultados - paginação, AI analysis, spell check, Hacker News indexing
        ├── links.html
        │   └── # Página de backlinks - lista URLs que apontam para um URL específico
        ├── stats.html
        │   └── # Dashboard de estatísticas - utiliza cliente WebSocket para updates em tempo real
        ├── googol_logo.svg
        │   └── # Logo (duplicado para templates)
        └── (sem templates adicionais mapeados)
```

```
config/
├── application.properties
│   └── # Configurações Spring Boot (RMI host/port, API keys, portas, profiles)
├── key.openrouterai.txt
│   └── # Chave da API OpenRouter.ai (opcional, pode estar em variáveis de ambiente)
└── system.properties
    └── # Propriedades do sistema (meta 1)
```

4. Integração com o servidor RPC/RMI (Meta 1)

4.1. Arquitetura de integração

A aplicação Web atua como cliente RMI do sistema distribuído desenvolvido na Meta 1. A comunicação é totalmente transparente para o utilizador Web, mas internamente utiliza RMI para todas as operações de indexação e pesquisa.

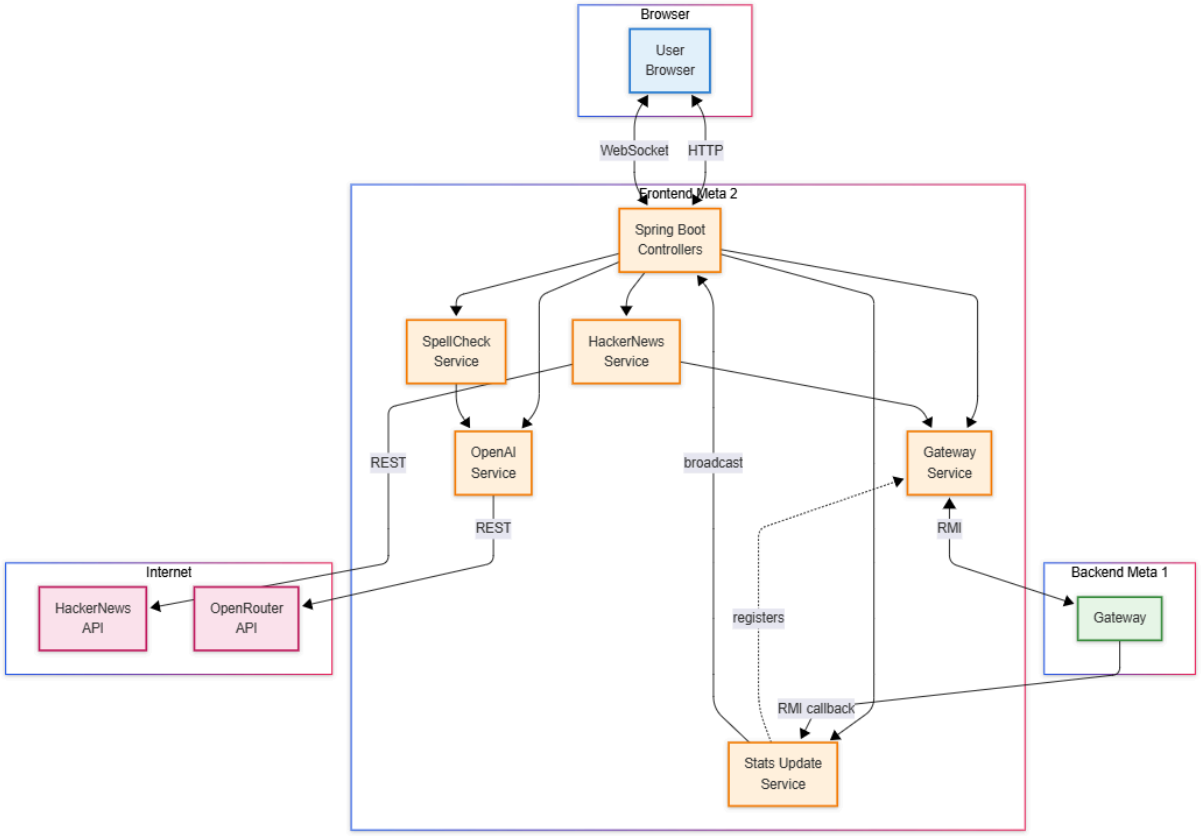


Figura 1 - Arquitetura de integração da aplicação web com o resto do sistema distribuído
Nota: após a elaboração do diagrama anterior foi implementado o uso de **HTTPS**.

4.2. *GatewayService*: Cliente RMI

A classe *GatewayService* é o único ponto de contacto entre o frontend e o backend RMI. Está programado de modo a que seja resiliente, ou seja, perante a perda de conexão com o Gateway, o frontend irá tentar a recuperação da conexão. É configurável a via *application.properties*, permitindo a facil configuração do host, porta e RMI registry host.

4.3. Fluxo de uma pesquisa

Apresenta-se de seguida o fluxo completo de uma pesquisa:

1. Browser → Spring Boot:
   Utilizador acede via *GET /search?q=pesquisa+exemplo&page=0*
2. SearchController:
   ```
   SearchResponse response = gatewayService.search(query, page);
   ```
3. GatewayService → Gateway RMI:
   ```
   return gateway.search(query, page); // Chamada RMI remota
   ```
4. Gateway (Meta 1):
   - Consulta barrels;
   - Barrels pesquisam no índice invertido;
   - Resultados são ordenados;
   - Retorna *SearchResponse*;
5. Spring Boot→ Thymeleaf:
   ```
   model.addAttribute("results", response.getResults());
   return "search"; // Renderiza search.html
   ```
6. Thymeleaf →Browser: HTML é renderizado com os resultados e é enviado ao browser.

4.4. Tratamento de falhas RMI

Perante uma falha na conexão com o gateway é lançada uma *RemoteException*. Desta forma, para garantir a validade da conexão com o gateway, existe o método *ensureConnected()*, chamado antes de todas as operações RMI, assegurando a validade das mesmas:

```
private void ensureConnected() throws RemoteException {
   if (gateway == null) {
      connect(); // Tenta conectar
   }

   try {
      gateway.getSystemStats(); // Testa conexão
   } catch (RemoteException e) {
```

```
        System.err.println("[GatewayService] Conexão perdida, a reconectar...");
        gateway = null;
        connect(); // Reconecta
    }
}
```

5. WebSockets e notificações em tempo real

Requisito: As estatísticas do sistema devem ser atualizadas em tempo real sem refresh periódico.
Solução: Arquitetura híbrida RMI Callback e WebSocket Push. O serviço *StatsUpdateService* efetua a ponte entre a comunicação RMI e via WebSocket.

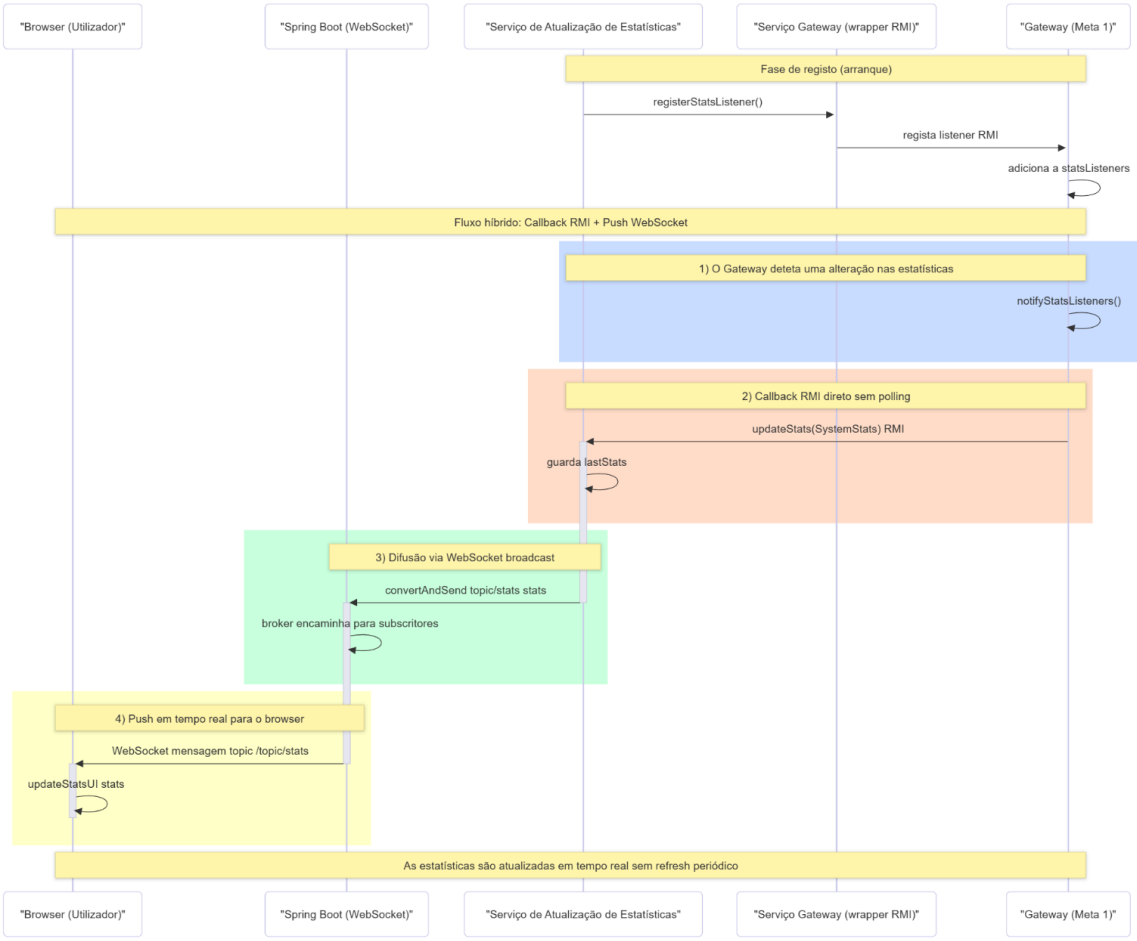


Figura 2 - Diagrama de sequência das estatísticas em tempo real

5.1. Configuração WebSocket (STOMP)

A classe WebSocketConfig configura o protocolo STOMP sobre WebSockets:

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

   @Override
   public void configureMessageBroker(MessageBrokerRegistry config) {
      config.enableSimpleBroker("/topic"); // Canal de broadcast
      config.setApplicationDestinationPrefixes("/app");
   }

   @Override
   public void registerStompEndpoints(StompEndpointRegistry registry) {
      registry.addEndpoint("/ws-stats")
            .setAllowedOriginPatterns("*")
            .withSockJS(); // Fallback para browsers antigos
   }
}
```

Características:

- **STOMP**: Protocolo de mensagens sobre WebSocket (simples e standard)
- **SockJS**: Fallback automático para long-polling se WebSocket não estiver disponível
- **Simple Broker**: Broker em memória para tópicos de broadcast (/topic/stats)

6. Integração com serviços REST externos

6.1. API HackerNews

Com o objetivo de permitir a indexação de top stories do HackerNews que contenham um conjunto de termos, a solução desenvolvida inclui um "modo de pesquisa" selecionável através de um seletor. Considerou-se que esta fosse a melhor abordagem, dado que a obtenção e pesquisa de termos nas top stories é uma tarefa custosa ao nível temporal, desta forma, o utilizador apenas submete um pedido de indexação que é processado de forma assíncrona e devidamente acrescentado à *UrlQueue* para posterior processamento pelos *Downloaders*.

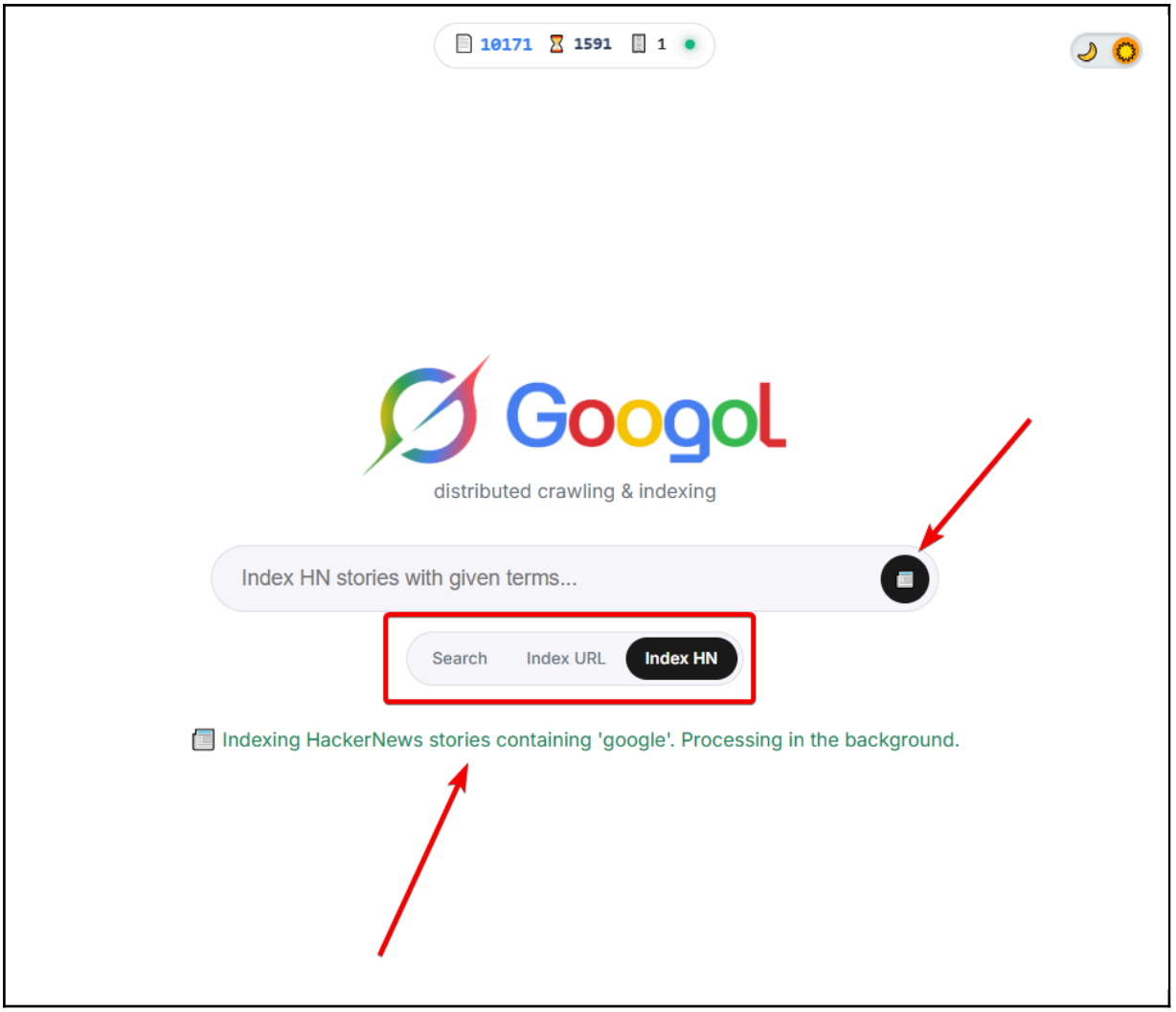


Figura 3 - Visualização da página inicial e do seletor de modos de funcionamento

6.1.1. Fluxo de indexação das top stories do HackerNews

1. Utilizador insere o termo de pesquisa
2. *SearchController* chama:

```
hackerNewsService.indexTopStoriesWithTermsAsync("<termos>", <maxurls>)
```

3. *HackerNewsService:*
   - 3.1. Efetua o pedido a: https://hacker-news.firebaseio.com/v0/topstories.json;
   - 3.2. Recebe array dos ids das *stories*;
   - 3.3. Para cada id faz o pedido a: /v0/item/{id}.json;
   - 3.4. Filtra as *stories* que contenham os termos inseridos;
   - 3.5. Para cada match, submete a *story* para indexação pelos *Downloaders*:

```
gatewayService.indexURL(story.url)
```

4. *Gateway* recebe os URLs via RMI e adiciona-os à fila com alta prioridade;
5. *Downloaders* processam o URL.

Características a destacar:

- O processo acontece de forma assíncrona: o processo de procura das top stories envolve o processamento de várias páginas com a finalidade de encontrar matches. Este processo é dispendioso e demorado. Como tal, o utilizador apenas submete o pedido e despoleta o processo, sem bloquear a utilização para o mesmo.
- Filtragem inteligente: apenas stories com os termos relevantes são indexadas;
- Integração transparente: do ponto de vista da Gateway, os URLs são tratados como qualquer outro enviado por outro cliente.

6.2. API Openrouter.ai

No lugar da API da OpenAI, para a utilização de LLMs no contexto do projeto recorreu-se aos modelos free da Openrouter.ai. Na implementação existem dois principais casos de uso para estes:

1. Análise contextualizada dos resultados de uma pesquisa;
2. Spellcheck;

6.2.1. Análise contextualizada dos resultados de uma pesquisa

Aquando da realização de uma pesquisa, surge um botão cuja finalidade é a submissão de um pedido para a análise contextualizada dos resultados. Com base nestes é oferecida uma análise simples de 3 parágrafos.

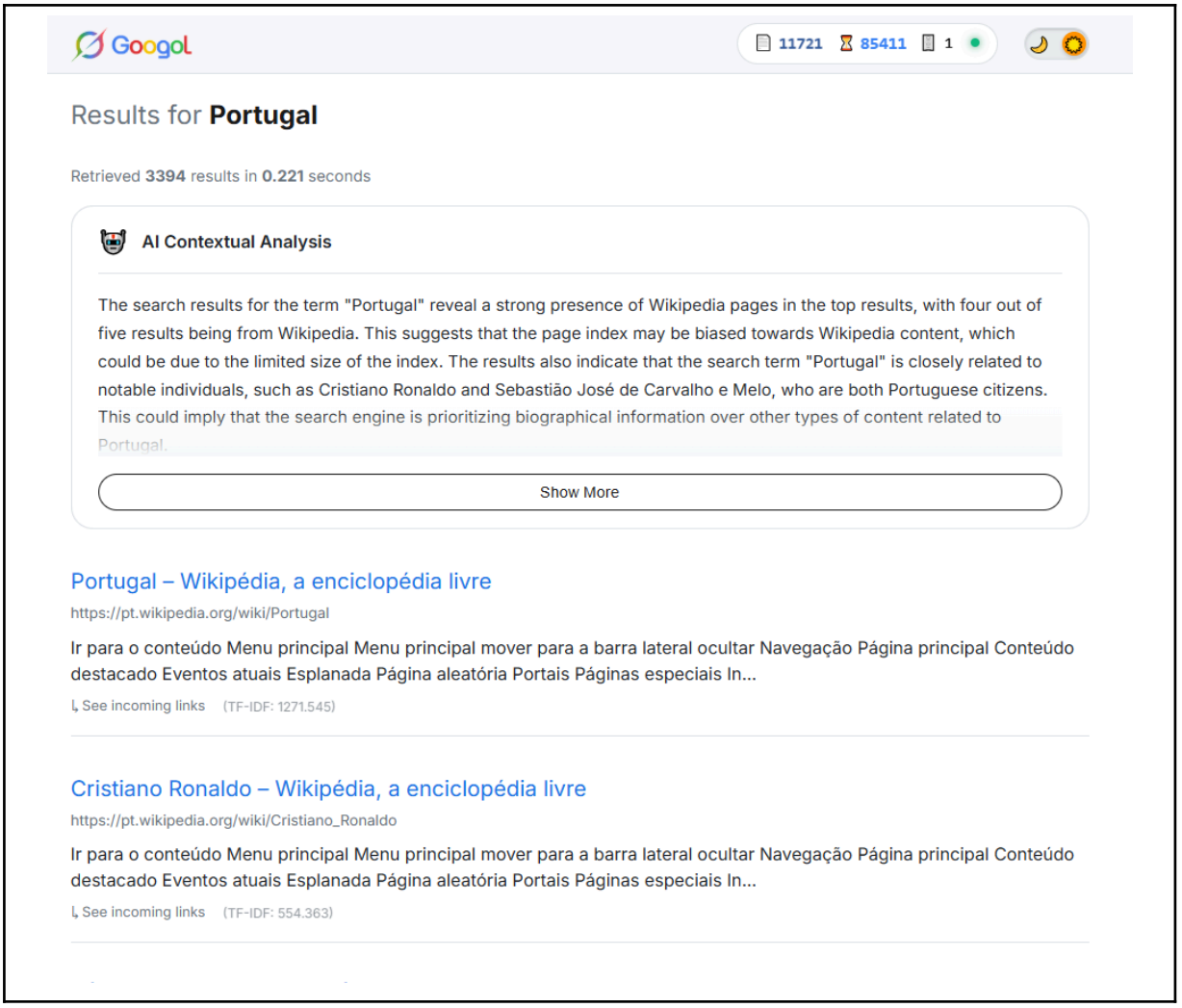


Figura 4 - Demonstração da análise contextualizada na página de resultados

6.2.1.1. Fluxo do processamento dos pedidos de análise contextualizada

1. Utilizador efetua uma pesquisa e pressiona o botão de *"AI Analysis"*;
2. *SearchController* trata do pedido e direciona-o para o *openAIService*;

```
if (aiAnalysis && results.size() > 0) {
    String analysis = openAIService.generateAnalysis(query, results);
    model.addAttribute("aiAnalysis", analysis);
```

3. *OpenAIService* constrói o prompt a ser submetido à API da *openrouter.ai*, incluindo os títulos e os snippets dos primeiros resultados;
4. *Openrouter.ai* recebe o pedido, processa-o com o modelo LLM selecionado e retorna os resultados;
5. Injeção da análise no template;
6. *Thymeleaf* renderiza a análise na secção destacada na página de resultados;

6.2.2. Spellcheck

Para enriquecer mais a experiência de pesquisa, decidiu-se implementar um spellchecker simples, recorrendo a prompts para LLMs.

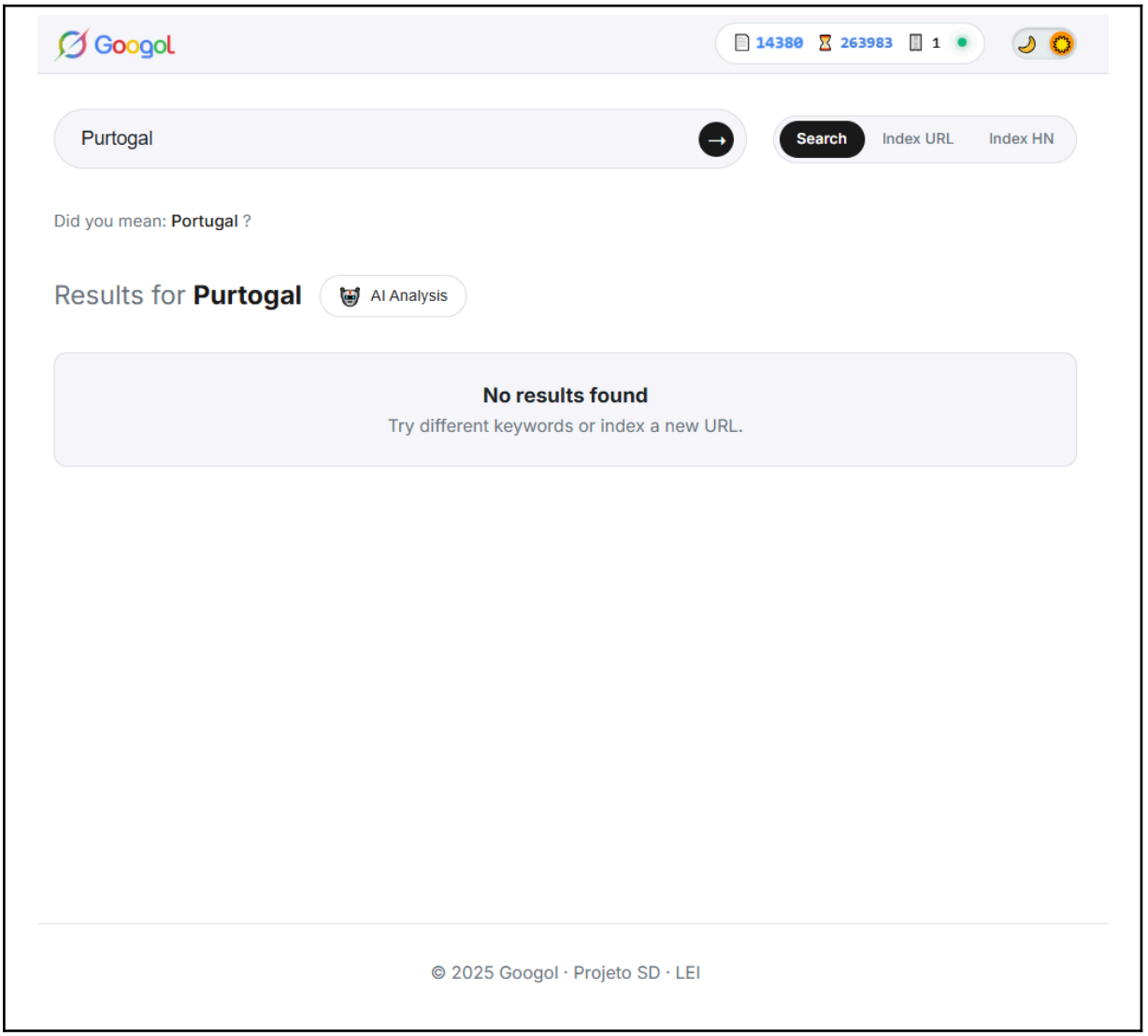


Figura 5 - Demonstração do funcionamento do spellcheck

De forma semelhante ao fluxo de processamento dos pedidos da análise contextualizada, o sistema submete cada pesquisa a uma validação por um modelo de inteligência artificial. Quando este entende que a pesquisa do utilizador pode ser refinada, ele oferece o termo de pesquisa que considera o desejado pelo utilizador, permitindo que este faça a pesquisa do termo potencialmente correto de forma mais fácil ao pressionar a sugestão.

7. Integração do projeto com um domínio web

Para explorar um cenário de deployment real da aplicação, adquiriu-se um domínio que efetua o redirecionamento para uma máquina a correr o SpringBoot. No entanto, como este exercício é apenas experimental, recorreu-se ao serviço de proxying gratuito da Cloudflare (Cloudflare Zero Trust) para restringir o acesso ao projeto e garantir a proteção da máquina a servir os pedidos (evitar a exposição para a internet toda). O domínio escolhido foi https://googol.projetosd.me, e garantiu-se a implementação de HTTPS quer do cliente para a Cloudflare como da Cloudflare para a máquina a correr o SpringBoot (já que a Cloudflare atua como proxy).

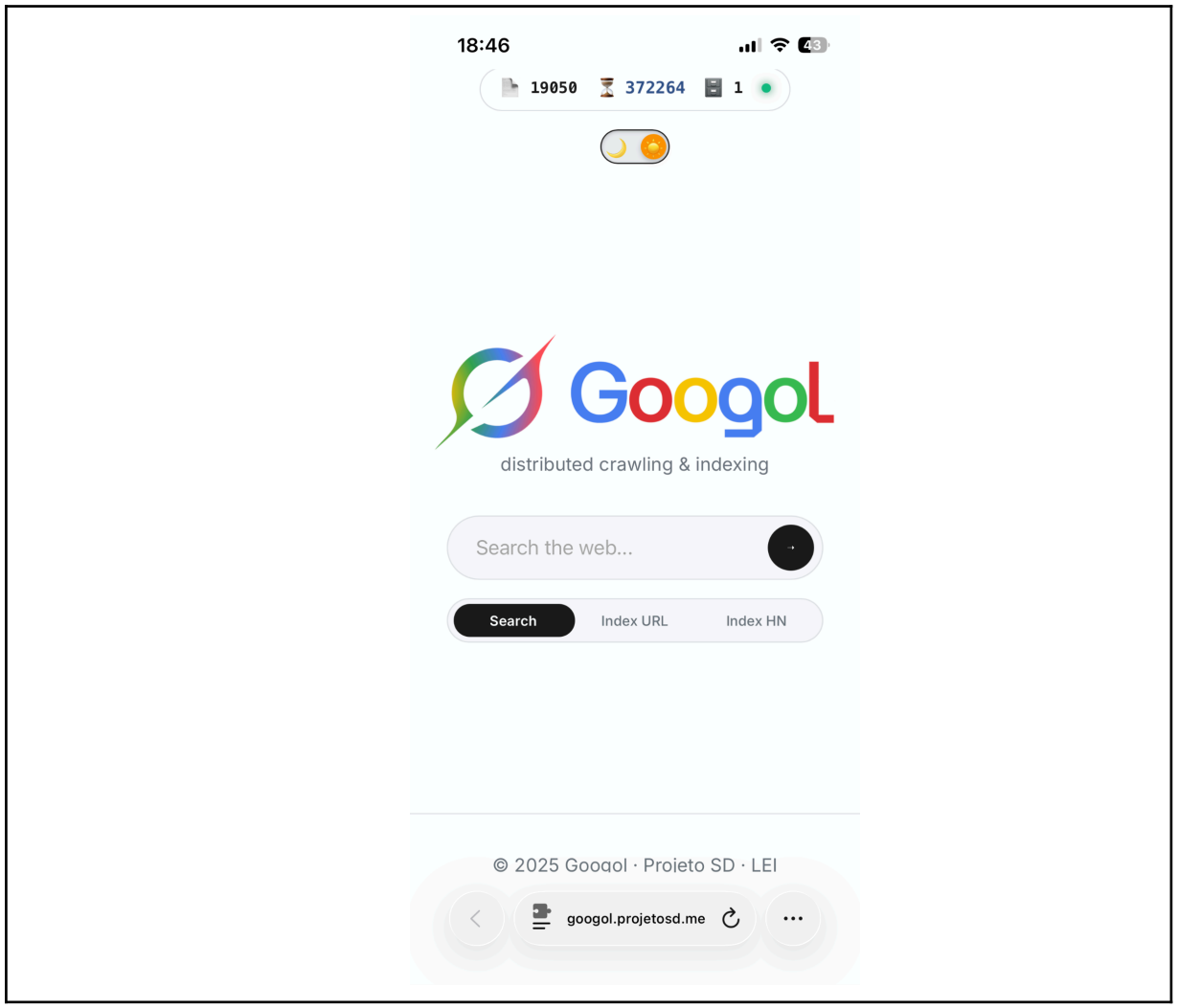


Figura 6 - Demonstração do acesso ao Googol através do domínio num dispositivo móvel

8. Testes de Software

Teste	Resultado
Indexar novo *URL* introduzido por utilizador	PASS
Indexar iterativamente ou recursivamente todos os *URLs* encontrados	PASS
Pesquisar todas as páginas que contenham um conjunto de palavras	PASS
Pesquisar páginas ordenadas por número de ligações recebidas de outras páginas	PASS
Consultar lista de páginas com ligações para uma página específica	PASS
Página de estatísticas atualizada em tempo real	PASS
Resultados de pesquisa agrupados de 10 em 10	PASS
Top 10 de pesquisas mais comuns atualizado em tempo real	PASS
Lista de barrels ativos com os respetivos tamanhos do índice	PASS
Tempos de resposta a pesquisas apresentados em décimas de segundo	PASS
Tratamento de vários clientes web em simultâneo	PASS
Indexar URLs de fonte externa (p.ex., HackerNews) contendo os termos da pesquisa	PASS
Gerar um texto contextualizado com IA (API REST de OpenAI, Gemini, Ollama, etc.)	PASS
Recuperação da conexão com a Gateway	PASS

Extras

Utilização de HTTPS	PASS
Utilização em smartphone ou tablet	PASS
Integração de um domínio web	PASS

9. Distribuição de tarefas

A distribuição das tarefas de desenvolvimento da presente meta foi feita de forma mais ou menos colaborativa da seguinte forma:

Diogo Rodrigues:
- Controladores MVC (*SearchController*, *ApiController*);
- WebSockets para estatísticas em tempo real;
- JavaScript client-side;

Guilherme Rosmaninho:
- Serviços de comunicação RMI (*GatewayService*);
- Integração com APIs externas (*HackerNews*, *Openrouter.ai*);
- Construção da interface visual (template e estilização);

10. Conclusão

A Meta 2 do projeto Googol resultou numa aplicação Web moderna e interativa. A utilização do Spring Boot simplificou a criação da arquitetura MVC e a integração de serviços REST. A combinação de Callbacks RMI com WebSockets provou ser uma solução eficaz para a atualização de dados em tempo real na página web, resultando no cumprimento de todos os requisitos operacionais e de experiência de utilizador propostos no enunciado.